

Rapport — Système de Gestion de Tickets

Présenté par : Dramé, Abdramane (DRAA04070300)

Section I — Critique du diagramme initial

Le diagramme de la partie 1 a plusieurs problèmes de conception qui rendent son implémentation difficile. D'abord, la classe Admin ne correspond pas au bon rôle. L'énoncé parle de développeurs qui assignent et qui ferment les tickets, mais le diagramme utilise une classe Admin qui n'apparaît jamais dans le texte. C'est le développeur qui gère les tickets, pas un admin. De plus, Admin et User sont séparés alors qu'un développeur EST un utilisateur. Un héritage aurait permis de réutiliser name et email et d'éviter la duplication. Ensuite, la classe Ticket fait trop de choses à la fois. Elle gère son identifiant, son titre, son statut, sa priorité, ses dates, ses descriptions, ses commentaires, son créateur et son assigné. Une classe qui fait autant de choses devient difficile à maintenir et à faire évoluer. Plusieurs méthodes sont aussi incohérentes. `viewAllTickets()` ne peut pas fonctionner sans liste centralisée, car la classe Admin ne possède aucun attribut pour stocker les tickets. `addComment()` ne peut rien conserver puisqu'aucun attribut comments n'est déclaré dans Ticket. De même, `assignTo()` ne peut pas mémoriser l'assignation car il manque un attribut `assignedTo`. Ces méthodes existent, mais elles ne peuvent pas faire leur travail. Les types utilisés sont également inadaptés. Le statut et la priorité sont des String, donc rien n'empêche d'écrire "ouvert", "Ouvert" ou "en cours" au lieu des valeurs valides. La description est aussi un simple String, ce qui ne permet pas de gérer du texte, des images et des vidéos avec des chargements différents, alors que l'énoncé le demande.

Enfin, il manque une centralisation des tickets. Sans classe dédiée, chaque partie du système doit trouver un moyen externe pour accéder aux tickets, ce qui crée un lien fort entre les classes.

Section II — Améliorations et justifications

Pour corriger ces problèmes, j'ai modifié le diagramme en appliquant les principes GRASP. Chaque changement vise à placer la responsabilité au bon endroit et à protéger le code contre les erreurs. Le tableau suivant résume les changements et leurs justifications.

Changement	Justification
Admin renommé en Developpeur	L'énoncé parle de développeurs, pas d'Admin. En appliquant Expert en information, c'est le développeur qui détient la connaissance pour assigner, valider et fermer un ticket.
Developpeur hérite de User	Un développeur EST un utilisateur spécialisé. Le Polymorphisme permet de réutiliser les méthodes de User et d'ajouter les siennes.
Classe abstraite DescriptionTicket	En appliquant Protégé des variations, on isole le chargement d'un texte, d'une image ou d'une vidéo du reste du code.
Sous-classes TextObject, ImageObject, VideoObject	Chaque sous-classe implémente load() selon son type. Le Polymorphisme permet d'ajouter un type sans modifier le reste du code.
DescriptionFactory ajoutée	En appliquant Fabrication pure, on crée une classe dédiée uniquement à la création des descriptions.
Statut remplacé par Enum	En appliquant Protégé des variations, seules les valeurs OUVERT, ASSIGNÉ, VALIDATION et TERMINÉ sont acceptées.
Priorite remplacé par Enum	Idem : seules les valeurs BASSE, NORMALE et HAUTE sont autorisées.
typeDescription remplacé par Enum	La factory reçoit un type sûr, impossible de passer un String invalide.
Commentaires retirés	L'énoncé n'en parle pas et TextObject gère déjà le texte. En appliquant Forte cohésion, on évite deux mécanismes redondants.
createdBy ajouté dans Ticket	En appliquant Expert en information, le ticket sait qui l'a créé, donc c'est à lui de le stocker.
assignedTo ajouté dans Ticket	Idem : le ticket doit savoir à qui il est assigné.
GestionnaireTickets ajouté	En appliquant Fabrication pure et Indirection, le gestionnaire centralise la liste et sert d'intermédiaire avec le Developpeur.

viewAllTickets(gestionnaire)	En appliquant Faible couplage, le Developpeur passe par le gestionnaire au lieu de dépendre directement de la liste.
role retiré de User	Avec l'héritage, le rôle est implicite. En appliquant Forte cohésion, on évite la redondance.

Section III — Autres améliorations du code

En plus des changements apportés au diagramme, j'ai ajouté plusieurs améliorations au code pour le rendre plus robuste et plus agréable à utiliser.

D'abord, j'ai mis en place une génération automatique des identifiants. Au lieu de demander à l'utilisateur de saisir l'ID du ticket ou de l'utilisateur, le programme génère un identifiant unique grâce à un compteur statique appelé `prochainId`. Cela évite les doublons et les erreurs de saisie. De la même façon, les dates de création et de mise à jour sont générées automatiquement avec `date.today()`, ce qui garantit qu'elles restent cohérentes et évite toute erreur humaine.

Ensuite, j'ai ajouté une gestion des cas vides. La méthode `viewAllTickets()` vérifie d'abord si le gestionnaire contient des tickets, et affiche un message clair sinon. Le programme principal fait la même vérification avant chaque action sur un ticket, ce qui évite les plantages. De plus, toutes les saisies utilisateur sont maintenant contrôlées grâce à des boucles `while`. On vérifie que le choix de menu est dans une liste valide, que le numéro de ticket est bien un chiffre, que le ticket existe réellement, que le type de description est valide, etc. Cela rend le programme beaucoup plus résistant aux erreurs de l'utilisateur. J'ai aussi amélioré l'assignation des tickets. Au lieu de demander à nouveau le nom et l'email du développeur, le programme affiche la liste des développeurs enregistrés et demande l'ID. Cela évite de créer un nouveau développeur à chaque assignation, ce qui aurait créé des doublons. Enfin, j'ai ajouté un lien bidirectionnel entre `User` et `Ticket`. Lorsqu'un ticket est assigné, la méthode `assignTo()` ajoute automatiquement ce ticket dans la liste de l'utilisateur. Ainsi, le ticket connaît son assigné, et l'utilisateur connaît ses tickets. La méthode `viewTicket()` affiche aussi toutes les descriptions du ticket en appelant `loadDescriptions()`, qui utilise le Polymorphisme pour charger chaque description selon son type.

Section IV — Leçons apprises

Au terme de ce travail, ce que l'on retient, c'est que le diagramme initial avait plusieurs problèmes qui ne respectaient pas l'énoncé ni les bonnes pratiques de

conception. En appliquant les principes GRASP, on a pu améliorer le diagramme pour supporter les pièces jointes, protéger les statuts avec des énumérations, et organiser le code de manière plus propre.

Les principes les plus importants dans ce travail sont l'Expert en information, la Fabrication pure, le Faible couplage et le Protégé des variations. En effet, l'Expert en information nous a permis de placer chaque responsabilité dans la bonne classe. La Fabrication pure nous a permis de créer des classes utilitaires comme Factory et Gestionnaire, sans surcharger les classes métier. Le Faible couplage nous a évité de dépendre directement de la liste des tickets. Enfin, le Protégé des variations nous a permis d'utiliser des énumérations et du polymorphisme pour rendre le code plus robuste. J'ai aussi appris qu'il faut toujours analyser l'énoncé avant de concevoir un diagramme. En effet, plusieurs problèmes venaient du fait que le diagramme initial ne respectait pas le texte. Une bonne conception commence toujours par une bonne compréhension du besoin. De plus, j'ai compris l'importance du polymorphisme et des design patterns comme la Factory. Ces outils permettent d'écrire du code plus propre et plus facile à faire évoluer. Par exemple, si on devait ajouter un type audio dans le futur, il suffirait de créer une sous-classe AudioObject et de l'ajouter dans la factory, sans toucher au reste du code. Enfin, ce travail m'a montré que la conception et le code doivent aller ensemble.